

CS5326 – Advanced GenAI and Agents
Fall 2026

Agha Ali Raza

Long-Context Extension



Required Readings

1. Sinusoidal positional encoding (the starting point)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.

<https://arxiv.org/abs/1706.03762>

Read: Section 3.5 only.

2. RoPE

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063. (arXiv preprint 2021.)

<https://arxiv.org/abs/2104.09864>

Read: Sections 3.2 to 3.4, including the long-term decay derivation.

3. Position interpolation

Chen, S., Wong, S., Chen, L., and Tian, Y. (2023). Extending Context Window of Large Language Models via Positional Interpolation. *arXiv preprint arXiv:2306.15595*.

<https://arxiv.org/abs/2306.15595>

Read: Section 2 and the interpolation bound in Section 3; skim experiments.

4. NTK-aware scaling

bloc97 (2023). NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation. Reddit, r/LocalLLaMA, June 2023.

https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/ntkaware_scaled_rope_allows_llama_models_to_have/

Note: a forum post, not a paper, but it is the original source and is cited as such by Meta (Code Llama) and Alibaba (Qwen).

5. Dynamic NTK scaling

emozilla (2023). Dynamically Scaled RoPE further increases performance of long context LLaMA with zero fine-tuning. Reddit, r/LocalLLaMA, June 2023.

https://www.reddit.com/r/LocalLLaMA/comments/14mrgpr/dynamically_scaled_rope_further_increases/

6. NTK-by-parts and YaRN

Peng, B., Quesnelle, J., Fan, H., and Shippole, E. (2024). YaRN: Efficient Context Window Extension of Large Language Models. *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*.

<https://arxiv.org/abs/2309.00071>

Read: all of Section 3. NTK-by-parts has no separate paper; Section 3.2 here is its formal statement.

7. ALiBi

Press, O., Smith, N. A., and Lewis, M. (2022). Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. *Proceedings of the Tenth International Conference on Learning Representations (ICLR 2022)*.

<https://arxiv.org/abs/2108.12409>

Optional Readings

1. Neural Tangent Kernel

Jacot, A., Gabriel, F., and Hongler, C. (2018). Neural Tangent Kernel: Convergence and Generalization in Neural Networks. *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*.

<https://arxiv.org/abs/1806.07572>

Read: Sections 1, 2, and 5 (the eigen-decomposition of training dynamics). The proofs can be skipped.

2. Fourier features and spectral bias, the paper the RoPE naming actually leans on

Tancik, M., Srinivasan, P. P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J. T., and Ng, R. (2020). Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.

<https://arxiv.org/abs/2006.10739>

The Central Question

How do we fit a **longer sequence** into a model's positional experience while **preserving useful distinctions between nearby tokens**?

A formula can outgrow its training

Defined at a new position does not mean learned behavior generalizes there.



Suppose we have already trained our model using RoPE with a context length of 4,000, but we want to support a context length of 12,000 for inference.

Let's start with four rotating hands of RoPE

- Assume

Symbol	Meaning	Our example
L	Original training context length	4,000 tokens
L'	Desired context length	16,000 tokens
$s = L' / L$	Context extension factor	4
d	Query/key dimension in one head, all rotated here	8
b	RoPE's frequency base	10,000
i	Coordinate-pair index	0,1,2,3

- An eight-dimensional query has four pairs:

$$q = \left(\underbrace{q_0, q_1}_{i=0}, \underbrace{q_2, q_3}_{i=1}, \underbrace{q_4, q_5}_{i=2}, \underbrace{q_6, q_7}_{i=3} \right)$$

- Each pair gets its own angular speed:

$$\omega_i = b^{-2i/d}$$

- That is** “Start with speed 1, then assign progressively smaller speeds to later pairs.”

Setting up Conventions

An eight-dimensional query has four pairs:

$$q = \left(\underbrace{q_0, q_1}_{i=0}, \underbrace{q_2, q_3}_{i=1}, \underbrace{q_4, q_5}_{i=2}, \underbrace{q_6, q_7}_{i=3} \right)$$

- Each pair gets its own angular speed:

$$\omega_i = b^{-2i/d}.$$

For $d = 8$:

$$\omega_i = 10000^{-i/4}$$

Because $10000 = 10^4$:

$$10000^{-i/4} = (10^4)^{-i/4} = 10^{-i}$$

So our four speeds are particularly simple:

Pair i	Angular speed ω_i , radians/token	Angle at position $p = 10$: $p\omega_i$
0	1	10 radians
1	0.1	1 radian
2	0.01	0.1 radians
3	0.001	0.01 radians

The angle formula is simply:

$$\underbrace{\phi_i(p)}_{\text{angle}} = \underbrace{p}_{\text{token position}} \underbrace{\omega_i}_{\text{radians per token}}$$

It is the rotational equivalent of **distance = time (s) × speed (m/s)**.

- For example, at position 10, a hand moving 0.1 radians/token has turned $10(0.1) = 1$ radian.

Why does relative position appear in RoPE?

For **one coordinate pair**, suppose the query and key initially point at angles a and b . RoPE adds their positional rotations:

$$\text{Query angle} = a + m\theta, \text{Key angle} = b + n\theta.$$

Their angular difference is therefore:

$$(a + m\theta) - (b + n\theta) = \underbrace{(m - n)\theta}_{\text{positional difference}} + \underbrace{(a - b)}_{\text{original content-angle difference}}.$$

Using **dot product = product of lengths $\times$ cosine of the angle between them**:

$$\boxed{\tilde{q}^\top \tilde{k} = \|q\| \|k\| \cos((m - n)\theta + (a - b))}$$

Using our example for RoPE

Query position: $m = 10$; **key position:** $n = 0$.

Original frequency: $\theta = 0.1$ radians/token.

Extension factor: $s = 4$, extending from 4,000 to 16,000 tokens.

Unrotated pairs: $q = k = (1, 0)$, so their lengths are 1 and initial angles are $a = b = 0$.

Original RoPE

The query and key rotate to:

$$\text{Query angle} = a + m\theta = 0 + 10(0.1) = 1,$$

$$\text{Key angle} = b + n\theta = 0 + 0(0.1) = 0.$$

Their dot product is:

$$\begin{aligned}\tilde{q}^\top \tilde{k} &= \|q\| \|k\| \cos((m - n)\theta + (a - b)) \\ &= 1(1) \cos((10 - 0)(0.1) + (0 - 0)) \\ &= \cos(1) \approx \boxed{0.5403}\end{aligned}$$

- This is the cosine of 1 radian.

Remember RoPE

Every token uses all four hands. These are four pairs within each Q/K vector. We'll just simplify below:

What does “rotate the pair” actually do?

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

Multiplying out:

$$x' = x \cos \phi - y \sin \phi, y' = x \sin \phi + y \cos \phi.$$

For the simple vector $(1, 0)$, this becomes:

$$(1, 0) \mapsto (\cos \phi, \sin \phi).$$

At $\phi = 1$:

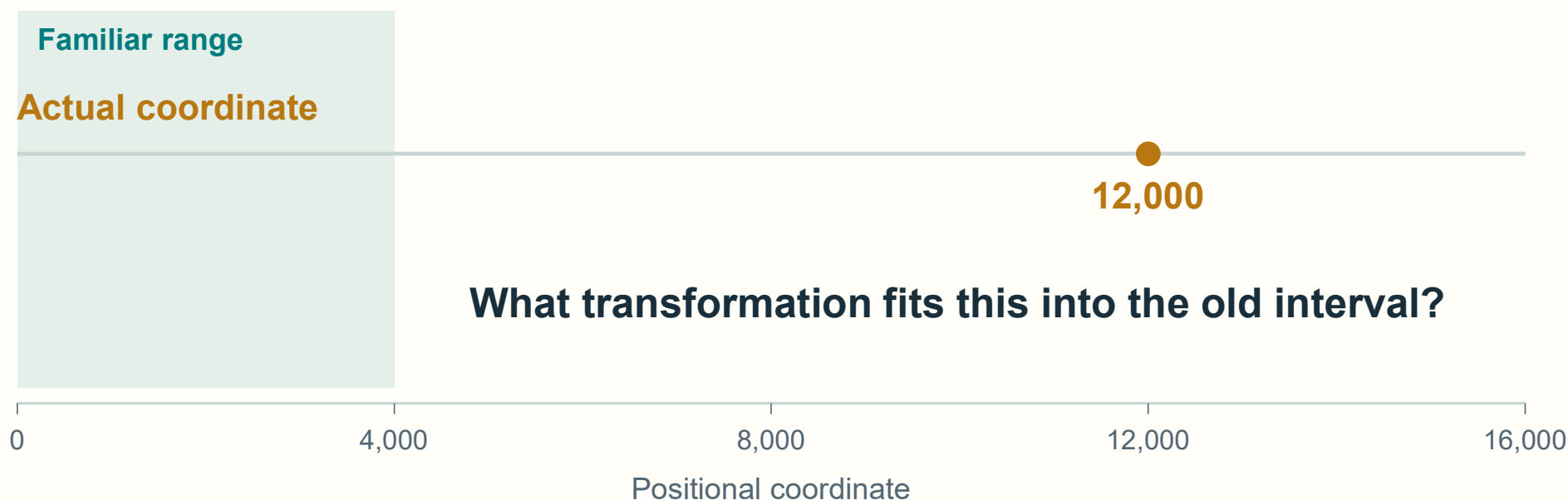
$$(1, 0) \mapsto (0.5403, 0.8415).$$

The arrow turns; its length remains 1.

POSITION INTERPOLATION

Fit the longer sequence onto the old scale

The target is 16,000 tokens. The familiar coordinate interval is 0–4,000.



Keep the tokens. Change the ruler used to assign their rotations.

Position Interpolation

The simplest fix: use $p/4$ in place of p .

$$\text{new angle} = \frac{p}{4} \omega_i$$

Thus, actual position 12,000 receives the rotation previously associated with position 3,000.

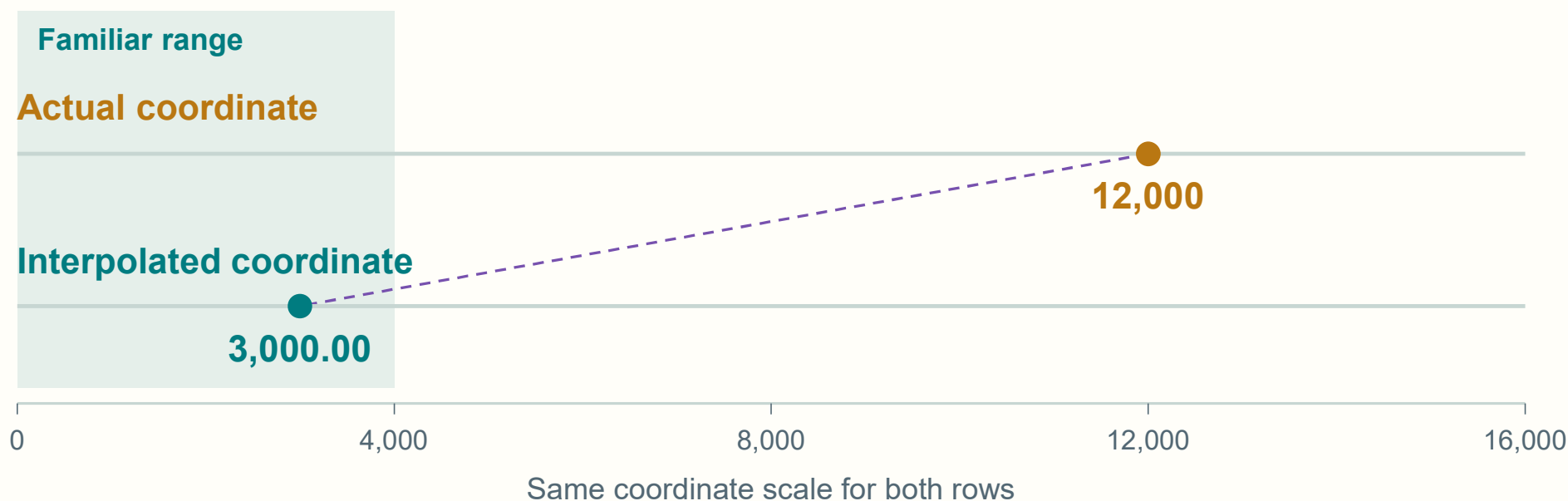
We still process all 16,000 tokens. We only compress their positional coordinates.

Equivalently, **slow every clock hand to one-quarter speed.**

POSITION INTERPOLATION

Compress positions, keep every token

A longer coordinate interval maps into the old one through $p' = p/s$.



The token count stays the same; only positional coordinates change.

POSITION INTERPOLATION

Slow all rotation speeds

Scaling position and scaling frequency are exactly the same operation.

Scale the position

$$(p / s) \times \omega$$

Scale the frequency

$$p \times (\omega / s)$$

=

Illustrative angular steps per token

Fast pair: $20^\circ \rightarrow 5.00^\circ$

Slow pair: $1^\circ \rightarrow 0.250^\circ$

Every frequency is divided by 4.

PI introduces a slowdown for the entire frequency spectrum.

Position interpolation: make every hand four times slower

Replace p with p/s :

$$\phi'_i(p) = \frac{p}{s} \omega_i = p \left(\frac{\omega_i}{s} \right).$$

The equality explains the two interpretations:

- **Coordinate interpretation:** divide the position by four.
- **Speed interpretation:** divide the frequency by four.

Do one or the other. Doing both would accidentally divide the angle by sixteen.

Actual token position p	Coordinate used by RoPE $p/4$
0	0
1	0.25
4	1
12,000	3,000
15,999	3,999.75

- We keep every token. We evaluate sine and cosine at more closely spaced positional coordinates.

Why does this help?

Consider the slowest pair, with $\omega = 0.001$.

- Across the original window, its angular span is approximately:

$$L\omega = 4000(0.001) = 4 \text{ radians.}$$

- Across the extended window without adjustment:

$$L'\omega = 16000(0.001) = 16 \text{ radians.}$$

- With interpolation:

$$L' \frac{\omega}{4} = 16000(0.00025) = 4 \text{ radians.}$$

The longer window now occupies the same angular span.

- Here and below, L denotes the conventional window-length span; actual zero-based positions end at $L-1$

What is the cost?

The same compression applies to nearby tokens (the query is at p , key is at j):

$$\text{relative angular separation} = \frac{p - j}{4} \omega_i.$$

For our earlier tokens, ten positions apart, with $\omega = 0.1$:

$$\text{original separation} = 10(0.1) = 1,$$

$$\text{interpolated separation} = 10(0.025) = 0.25.$$

For the illustrative $q = k = (1, 0)$:

$$\text{original contribution} = \cos(1) \approx 0.5403,$$

$$\text{new contribution} = \cos(0.25) \approx 0.9689.$$

The contribution changes substantially because the positional rotation is smaller.

- The tokens have come “closer”
- This does **not** mean interpolation always increases attention: the result also depends on Q/K content.

The tradeoff is that large distances become more familiar.

Mathematically: Relative position in RoPE and Position Interpolation

For **one coordinate pair**, suppose the query and key initially point at angles a and b . RoPE adds their positional rotations:

$$\text{Query angle} = a + m\theta, \text{Key angle} = b + n\theta.$$

Their angular difference is therefore:

$$(a + m\theta) - (b + n\theta) = \underbrace{(m - n)\theta}_{\text{positional difference}} + \underbrace{(a - b)}_{\text{original content-angle difference}}.$$

For RoPE, Using **dot product = product of lengths × cosine of the angle between them**:

$$\tilde{q}^\top \tilde{k} = \|q\| \|k\| \cos \left((m - n)\theta + (a - b) \right)$$

Position Interpolation:

$$\cos \left(\underbrace{\frac{m - n}{s} \theta}_{\text{compressed positional difference}} + \underbrace{(a - b)}_{\text{content-angle difference unchanged}} \right)$$

Using our example for RoPE and Position Interpolation

Query position: $m = 10$; **key position:** $n = 0$. , **Original frequency:** $\theta = 0.1$ radians/token.

Extension factor: $s = 4$, extending from 4,000 to 16,000 tokens. , **Unrotated pairs:** $q = k = (1, 0)$.

Original RoPE: The query and key rotate to:

$$\text{Query angle} = a + m\theta = 0 + 10(0.1) = 1,$$

$$\text{Key angle} = b + n\theta = 0 + 0(0.1) = 0.$$

Their dot product is:

$$\begin{aligned}\tilde{q}^\top \tilde{k} &= \|q\| \|k\| \cos((m - n)\theta + (a - b)) \\ &= 1(1) \cos((10 - 0)(0.1) + (0 - 0)) \\ &= \cos(1) \approx \boxed{0.5403}\end{aligned}$$

Position interpolation: Compress the positional difference by $s = 4$:

$$\begin{aligned}\tilde{q}^\top \tilde{k} &= \cos\left(\frac{m - n}{s}\theta + (a - b)\right) \\ &= \cos\left(\frac{10 - 0}{4}(0.1) + (0 - 0)\right) \\ &= \cos(0.25) \approx \boxed{0.9689}.\end{aligned}$$

The query now rotates by 0.25 radians and the key by 0. Their **actual sequence positions remain 10 and 0**.

Question

Is PI applied after the model has already been trained using RoPE?

Yes. The original method starts from a pretrained RoPE model, changes the positional mapping, and then may continue training with that mapping. The modified mapping remains in use during inference.

Fine-tuning on what exactly?

On text sequences using ordinary next-token prediction, with the modified positional encoding active.

- The original PI work used a pretraining corpus such as the Pile and extended context windows.

A concrete training procedure is:

1. Load the pretrained model.
2. Change RoPE to use $p/4$.
3. Feed long text sequences, for example up to 16,000 tokens.
4. Predict each next token from the preceding tokens.
5. Update the model's learned weights through backpropagation.

The training signal is the text itself. Human-written question–answer labels are unnecessary.

POSITION INTERPOLATION

More range means tighter local spacing

The same compression also changes the geometry of neighboring tokens.

Original positional spacing



After 4× extension



100 tokens → 25.0 old positional units

The high Frequencies did not need a slow-down! What now?

Frequency-dependent scaling: “Why slow every hand equally?”

- Fast hands give substantial angular changes between nearby tokens, useful for distinguishing local distances. Slow hands change gradually over longer spans.

So we can **preserve fast rotations more, while slowing slow rotations more**.

- Picture a clock: preserve the second hand’s fine detail while extending the hour hand’s range.
- “Frequency-dependent scaling” names a **family of choices** about how much to change each frequency. YaRN uses one particular choice.

- YaRN Paper: <https://arxiv.org/pdf/2309.00071>

FREQUENCY-DEPENDENT SCALING

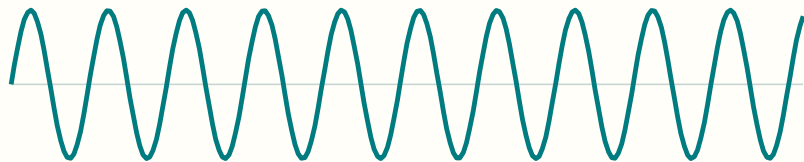
Different speeds capture different scales

Fast pairs change over short spans. Slow pairs change over longer spans.

FAST PAIR · $i = 0$

$\omega = 1$ rad/token

One turn every 6.28 tokens



Same 64-token span

SLOW PAIR · $i = 28$

$\omega \approx 0.000316$ rad/token

One turn every 19,869 tokens



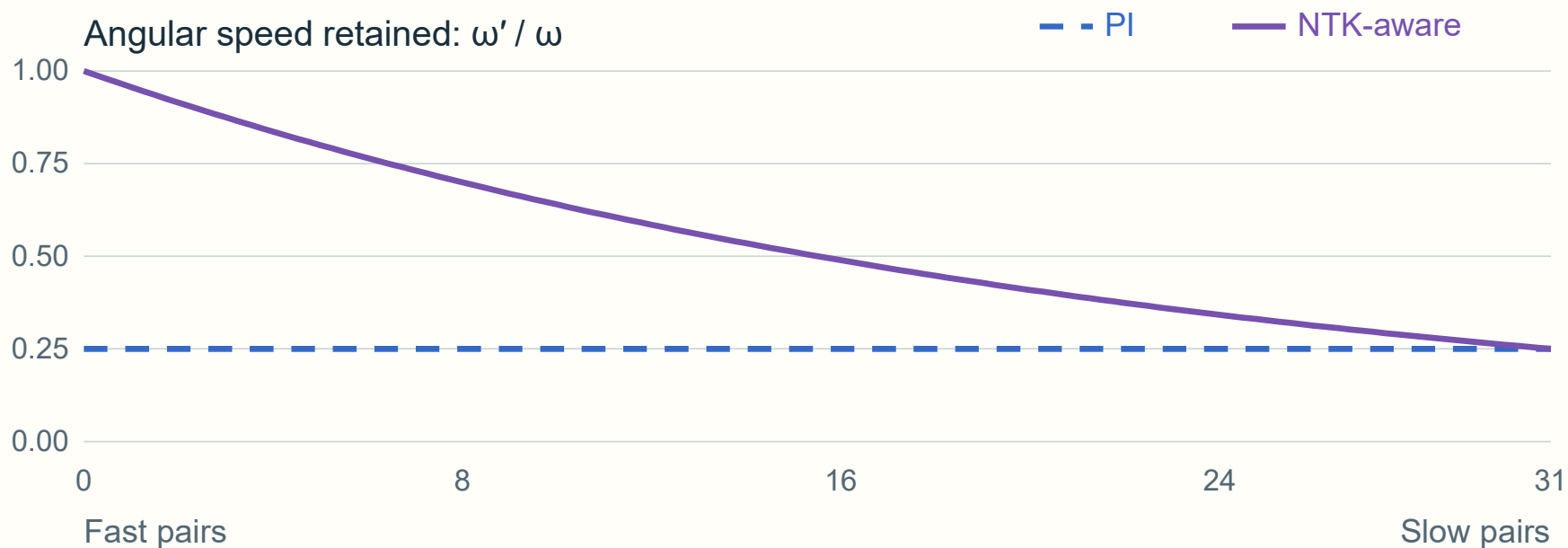
Same 64-token span

Use this as an intuition about scale, not a rule limiting what each pair can learn.

FREQUENCY-DEPENDENT SCALING

Change the base, vary the slowdown

A larger RoPE base reduces slow frequencies more than fast ones.



The fastest pair stays unchanged; the slowest reaches the full 1/s slowdown.

Inspect what happens to one pair

Compare equal treatment under PI with selective treatment under NTK-aware scaling.

Pair 16 · original $\omega = 0.01000$ rad/token

Method	Speed retained	New angular step
Original RoPE	100%	0.010000
PI	25.0%	0.0025000
NTK-aware	48.9%	0.0048894

A shared extension target does not require a shared slowdown for every pair.

NTK-aware scaling: let different hands slow by different amounts

NTK stands for Neural Tangent Kernel. It is a mathematical framework describing neural-network learning through how parameter changes affect predictions across inputs.

You do not need to calculate an NTK to apply this RoPE scaling rule.

NTK-aware scaling: let different hands slow by different amounts

The basic idea:

$$\omega_{\text{new}} = (\omega_{\text{old}})^p, p > 1$$

- **Our fastest** $\omega_{\text{old}} = 1$.
- **Use the same exponent p for every frequency.**
- Larger p means stronger scaling.

For example, choosing $p = 2$:

- Fast: $1 \rightarrow 1$, **unchanged**.
- Medium: $0.1 \rightarrow 0.01$, **10 times slower**.
- Slow: $0.01 \rightarrow 0.0001$, **100 times slower**.

This is mathematically equivalent to NTK-aware base scaling.

- **One shared exponent automatically slows smaller frequencies proportionally more.**
- The choice $p = 2$ is just an illustration; it does not mean doubling the context length.

NTK-Aware Scaling

The key is: a larger pair index i means both a slower original frequency and a stronger effect from increasing the base.

Write the frequencies as fractions:

$$\omega_i = \frac{1}{b^{2i/d}}, \omega'_i = \frac{1}{(b')^{2i/d}}.$$

If we increase the base by a factor $k = b'/b > 1$, then:

$$\boxed{\frac{\omega'_i}{\omega_i} = \frac{1}{k^{2i/d}}}$$

As i increases, the denominator grows, so a smaller fraction of the original speed remains.

Using our $d = 8$ example, suppose:

- $b = 10,000$,
- $b' \approx 63,496$,
- hence $k \approx 6.35$:

- Thus, “**slower frequencies shrink more**” means a **greater percentage reduction**, not a larger absolute subtraction.
- Factor: $k = b'/b > 1$, then:

$$\frac{\omega'_i}{\omega_i} = \frac{1}{k^{2i/d}}$$

Pair i	Original speed	Fraction retained $1/6.35^{i/4}$	Reduction
0: fastest	1	$1/6.35^0 = 1$	0%
1	0.1	$1/6.35^{0.25} \approx 0.63$	37%
2	0.01	$1/6.35^{0.5} \approx 0.397$	60.3%
3: slowest	0.001	$1/6.35^{0.75} = 0.25$	75%

NTK-aware scaling: let different hands slow by different amounts

The relevant motivation is to preserve high-frequency features that help represent rapid positional variation. This motivated changing RoPE's base instead of dividing all frequencies equally.

The general frequency-dependent idea is:

$$\omega'_i = \frac{\omega_i}{s_i}.$$

- Here, s_i is the slowdown for pair i .
- For example, $s_i = 2$ turns 0.1radians/token into 0.05.

NTK-aware scaling obtains different slowdowns through one base change:

$$\omega'_i = (b')^{-2i/d} = 1/b'^{(2i/d)}$$

Divide by the original frequency:

$$\frac{\omega'_i}{\omega_i} = \frac{(b')^{-2i/d}}{b^{-2i/d}} = \left(\frac{b'}{b}\right)^{-2i/d}.$$

This ratio tells us what fraction of the old speed survives.

- A ratio of 1 means unchanged; 0.5 means half speed; 0.25 means quarter speed.

How do we choose b' ?

One standard static NTK-aware construction sets:

$$\boxed{b' = b s^{d/(d-2)}}.$$

- It is designed to leave the fastest pair unchanged and slow the slowest pair by exactly s .
- This is a construction rule, not a guarantee of a useful s -fold context extension.

For our example:

$$b' = 10000 \times 4^{8/6} \approx 63,496.$$

Substituting this base ratio into the frequency ratio:

$$\begin{aligned} \frac{\omega'_i}{\omega_i} &= \left(s^{d/(d-2)}\right)^{-2i/d} \\ &= s^{-2i/(d-2)} \\ &= 4^{-i/3}. \end{aligned}$$

How do we choose b' ?

Now calculate each pair:

Pair i	Original ω_i	Fraction retained $4^{-i/3}$	New ω'_i	Slowdown
0	1	1	1	$1 \times$
1	0.1	0.6300	0.06300	$1.59 \times$
2	0.01	0.3969	0.003969	$2.52 \times$
3	0.001	0.25	0.00025	$4 \times$

For pair 1 specifically:

$$\frac{\omega'_1}{\omega_1} = \left(\frac{63496}{10000}\right)^{-1/4} \approx 0.6300.$$

Therefore:

$$\omega'_1 = 0.1(0.6300) = 0.06300.$$

The fast end stays fast; the slow end gains the full extension; the middle changes gradually.

How do we choose b' ?

The fast end stays fast; the slow end gains the full extension; the middle changes gradually.

To see why the exponent $d/(d-2)$ appears, note that the last pair is $i = (d-2)/2$. We want:

$$\left(\frac{b'}{b}\right)^{-(d-2)/d} = \frac{1}{s}.$$

Taking reciprocals and then raising both sides to $d/(d-2)$:

$$\frac{b'}{b} = s^{d/(d-2)}.$$

That gives the stated base formula.

YaRN

Preserve local detail, extend the range, and adjust attention's sharpness

YaRN combines:

- **Selective frequency scaling:** leave fast rotations unchanged, slow the slow ones, and smoothly blend between these treatments for intermediate frequencies.
- **Attention scaling:** adjust scores before softmax to control how concentrated attention becomes.

Intuitively, a longer text introduces more tokens competing for attention.

- Adjusting score scale helps maintain useful focus; it does not identify relevant tokens by itself.
- **YaRN = a particular frequency-scaling recipe + an attention-temperature adjustment.**
 - Its demonstrated large extensions also involve a relatively small amount of fine-tuning.

Using our example for RoPE and YaRN

Query position: $m = 10$; **key position:** $n = 0$.

Original frequency: $\theta = 0.1$ radians/token.

Extension factor: $s = 4$, extending from 4,000 to 16,000 tokens.

Unrotated pairs: $q = k = (1, 0)$, so their lengths are 1 and initial angles are $a = b = 0$.

Original RoPE: The dot product is:

$$\begin{aligned}\tilde{q}^\top \tilde{k} &= \|q\| \|k\| \cos((m-n)\theta + (a-b)) \\ &= 1(1) \cos((10-0)(0.1) + (0-0)) \\ &= \cos(1) \approx \boxed{0.5403}.\end{aligned}$$

NTK-aware scaling and YaRN

This is pair $i = 1$ in our $d = 8$, $b_{\text{RoPE}} = 10,000$ example. Using the frequencies we calculated earlier:

$$q^\top k = \cos((10-0)\theta_1' + (0-0))$$

Method	Frequency θ_1'	Angular difference	Pair's dot product from rotation
Original RoPE	0.1	1	$\cos(1) \approx 0.5403$
Position interpolation	0.025	0.25	$\cos(0.25) \approx 0.9689$
NTK-aware	0.06300	0.6300	$\cos(0.6300) \approx 0.8080$
YaRN	0.1	1	$\cos(1) \approx 0.5403$

- YaRN preserves this pair's frequency because it completed approximately **63.66 revolutions** across the original window, above the preservation threshold of 32.
- The YaRN row shows its **rotation component**. Its additional Q/K length scaling multiplies this contribution by $c^2 \approx 1.29648$:
- $1.29648\cos(1) \approx \boxed{0.7005}$.
- These are contributions from **one coordinate pair**. Full attention sums all pairs' contributions, divides by $\sqrt{d}$, and applies softmax across eligible keys.

Why does relative position appear?

For one pair:

$$\begin{aligned}(R(p\omega)q)^\top (R(j\omega)k) &= q^\top R(p\omega)^\top R(j\omega)k \\ &= q^\top R(-p\omega)R(j\omega)k \\ &= q^\top R((j-p)\omega)k.\end{aligned}$$

Each step has a geometric meaning:

- Transposing a rotation matrix reverses its rotation.
- Two successive rotations add their angles.
- Therefore, the comparison retains the **difference between positions**.
- The **query is at position $p = 10$** and the **key is at position $j = 0$** , so their distance is 10 tokens.

For $p = 10, j = 0, \omega = 0.1$, and $q = k = (1, 0)$:

- pair contribution = $\cos((0 - 10)0.1) = \cos(-1) \approx 0.5403$.
- Moving both tokens forward by 100 positions gives $p = 110, j = 100$, and the same result.
- This isolates the positional effect while holding Q/K content fixed. A real attention dot product sums the contributions of all four pairs.

A useful idea still needs a precise rule

Which frequencies should stay fixed, which should slow, and how much?

1 Which rotation speeds should change?

Count turns across the original training window.

2 How concentrated should attention be?

Adjust how strongly score differences matter.

YaRN makes that choice explicit and separately adjusts attention concentration.

Count the turns seen during training

Use the original context length to characterize each pair's exposure.

Pair 16 · $\omega = 0.01000$

$$r = 4,000 \times \omega / (2\pi) = 6.366 \text{ turns}$$

Original revolutions

Treatment

$r \leq 1$

Fully interpolate

$1 < r < 32$

Blend the frequencies

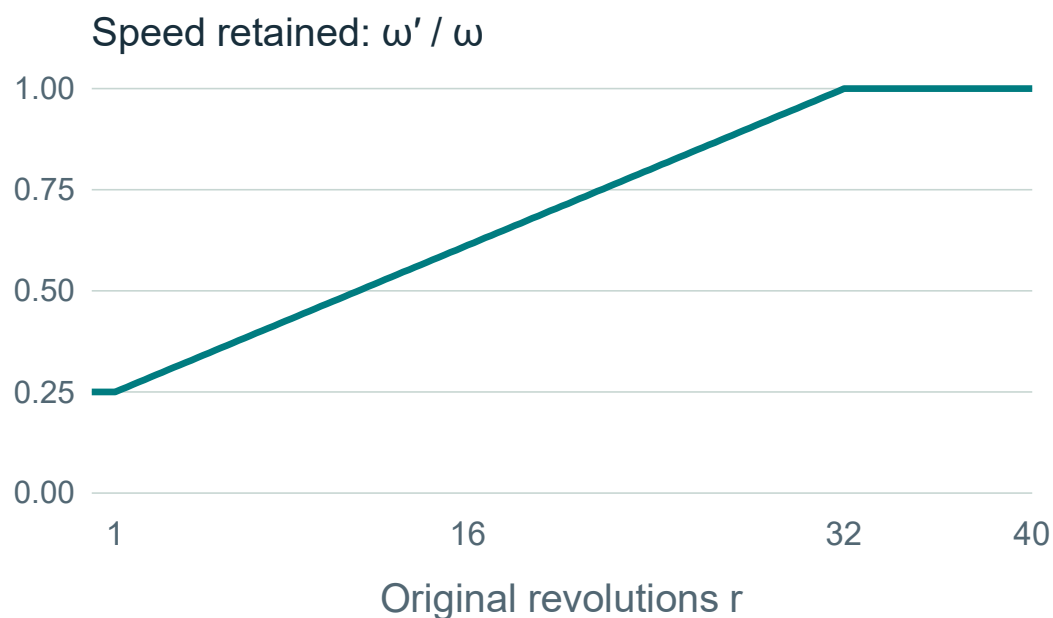
$r \geq 32$

Preserve the frequency

Few turns: interpolate. Many turns: preserve. Between them: blend.

Blend the original and slowed speeds

The preservation weight γ moves smoothly between two frequency choices.



$$\gamma = \text{clip}((r - 1)/31, 0, 1)$$

$$\omega' = (1 - \gamma)\omega/s + \gamma\omega$$

$\gamma = 0 \rightarrow$ full slowdown

$\gamma = 1 \rightarrow$ unchanged

Here: $s = 4$

$\gamma = 0$ gives full interpolation; $\gamma = 1$ keeps the original frequency.

Work one pair through the rule

Turn the definition into a reproducible numerical example.

1 · Original speed

$$\omega = 10,000^{(-2 \times 16 / 64)} = 0.010000$$

2 · Training turns

$$r = 4,000 \times \omega / (2\pi) = 6.3662$$

3 · Preservation weight

$$\gamma = \text{clip}((r-1)/31, 0, 1) = 0.1731$$

4 · New speed

$$\omega' = [(1-\gamma)/4 + \gamma] \omega = 0.0037983$$

One pair, one explicit calculation: original speed → turns → weight → new speed.

More keys compete for attention

Even unchanged content scores share their probability mass with more candidates.

One highlighted score = 2.5; every other score = 0.

3 other tokens



31 other tokens



Highlighted token

All other tokens combined

$$P(\text{highlighted}) = \exp(2.5) / [\exp(2.5) + N]$$

Frequency scaling changes geometry; it does not directly set softmax concentration.

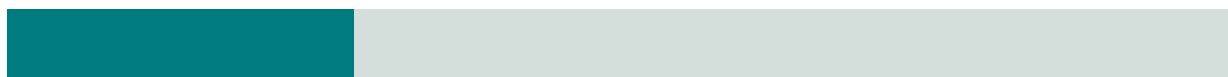
Sharpen preferences already in the scores

YaRN adds an empirical temperature adjustment to its frequency rule.

$$c = 1 + 0.1 \ln(4) = 1.139 \quad t = 1/c^2 = 0.771$$

31 competing tokens; highlighted score advantage = 2.5

Ordinary softmax



28.2%

YaRN score scaling



45.2%

Multiply Q and K by $c \rightarrow$ multiply the score by c^2 .

Scaling strengthens an existing preference. Equal scores remain equal.

ALiBi takes a different branch

Distance can enter the attention score directly as an additive bias.

RoPE family

Rotate projected Q and K

$$\mathbf{q}^T \mathbf{R}((\mathbf{j}-\mathbf{p})\omega) \mathbf{k}$$

Position changes the geometry

ALiBi

Bias the content score

$$\mathbf{q}^T \mathbf{k} / \sqrt{d - m(\mathbf{p}-\mathbf{j})}$$

Distance adds a recency preference

ALiBi is an alternative positional design, not the next RoPE extension.

Charge attention a distance fee

Subtract slope × distance before applying softmax.

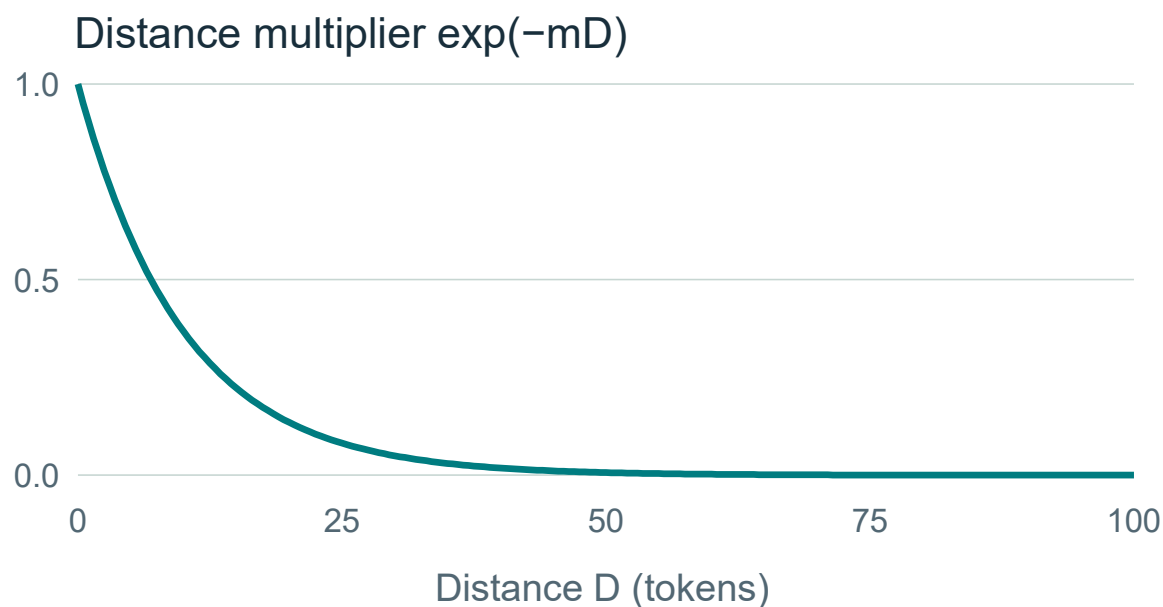
Token distance	Content score	Penalty	Final score
Near: 2	6.0	-0.20	5.80
Far: 20	6.0	-2.00	4.00

Head slope $m = 0.10$ per token

With equal content scores, the nearer token gets the higher adjusted score.

A linear fee gives an exponential discount

Exponentiating the adjusted score reveals the hidden multiplication.



$$\exp(u - mD)$$
$$= \exp(u) \times \exp(-mD)$$

$$m = 0.10$$

A multiplier, not a probability.

Unnormalized support = content support × distance discount.

Strong content can overcome distance

A farther token wins once its content advantage exceeds its extra penalty.

Extra distance penalty: $0.1 \times (20-2) = 1.8$

Far-token content advantage: 0.0 score units

Near token



Far token



The nearby token wins after the distance penalty.

The recency preference is a bias, not a hard ban on distant information.

COMPARISON

Match the mechanism to the problem

Each method changes a specific part of the positional computation.

Method	Exact change	Purpose
RoPE	Rotate Q and K	Relative positional geometry
PI	Divide every frequency by s	Extend coordinate range
Selective scaling	Vary the slowdown by pair	Preserve more local detail
YaRN	By-parts rule + score scaling	Geometry and concentration
ALiBi	Subtract $m \times \text{distance}$	Explicit recency preference

A longer coordinate range still needs learned behavior and affordable computation.

**Let's take an example through all the
techniques**

Assume

- Head dimension d (say 128) gives $\frac{d}{2} = 64$ pairs
- Pair i turns $\theta_i = 10000^{-\frac{2i}{d}}$ radians per token
- Its wavelength, the number of *tokens for one full turn*, is

$$\lambda_i = \frac{\text{number of radians in a full turn}}{\text{number of radians per token}} = \frac{2\pi}{\theta_i}$$

- The one number that matters for every method is *how many turns* pair i completed **during training**:

$$r_i = \frac{\text{number of tokens in training}}{\text{number of tokens in one full turn}} = \frac{L}{\lambda_i} = L \cdot \frac{\theta_i}{2\pi}$$

Making it concrete

For LLaMA-style $L = 4096$:

- Pair 0
 - has $\theta = 1$ *radians per token*,
 - $\lambda \approx 6.3$ tokens, $r \approx 650$ turns.
- Pair 63
 - has $\theta \approx 1.15 \times 10^{-4}$ *radians per token*
 - $\lambda \approx 54,000$ tokens, $r \approx 0.075$ turns.
- Solve $\lambda_i = L$ for the crossover:
 - $\theta_i = \frac{2\pi}{L} \approx 1.5 \times 10^{-3}$, which lands at $i \approx 45$.

So the first 45 or so pairs saw the full circle many times; the last 18 or so never completed one turn.

Those 18 are where extrapolation dies.

Worked out i

Set the wavelength of pair i equal to the training length, then solve for i .

Step 1. The condition. Pair i completes exactly one turn during training when

$$\lambda_i = L \iff \frac{2\pi}{\theta_i} = L \iff \theta_i = \frac{2\pi}{L} = \frac{2\pi}{4096} \approx 1.534 \times 10^{-3}$$

Step 2. Plug in the RoPE frequency formula with $d = 128$:

$$\theta_i = 10000^{-\frac{2i}{128}} = 10^{-4 \cdot \frac{2i}{128}} = 10^{-\frac{i}{16}}$$

- Base 10000 is 10^4 , so the exponent simplifies nicely: each pair index divides the frequency by $10^{1/16}$)

Step 3. Solve.

$$\begin{aligned} 10^{-\frac{i}{16}} &= 1.534 \times 10^{-3} \\ -\frac{i}{16} &= \log_{10}(1.534 \times 10^{-3}) = 0.186 - 3 = -2.814 \\ i &= 16 \times 2.814 \approx 45.0 \end{aligned}$$

Main Idea of the next step

RoPE

Encodes position by rotating each query and key pair (i.e., each circle) by an angle proportional to its position, so the attention score depends only on the distance between tokens.

Downside: past the training length, the slow pairs reach angles they never saw, and attention collapses within a few hundred tokens.

RoPE: why "unseen angle" is the precise failure

For pair i , the score contribution is $|q||k| \cdot \cos(\alpha_i - \beta_i + (m - n)\theta_i)$

Training only ever fed the positional term values $(m - n)\theta_i \in [0, L \cdot \theta_i]$

- For fast pairs $L \cdot \theta_i \gg 2\pi$, so modulo 2π they covered every angle.
- For slow pairs $L \cdot \theta_i < 2\pi$, so they only ever saw a small arc, and Wq, Wk shaped α_i, β_i under the assumption that the positional offset stays in that arc.
- During training, a slow pair barely rotates, so the model quietly uses it as a near-absolute channel ("this key is early in the document").
 - Push $m - n$ past L and that channel is suddenly asked to report an angle it has no learned meaning for.
 - The score is arbitrary.
 - That is why perplexity blows up within a few hundred tokens past L , not gradually.

Evidence of Extrapolation Failure

The following figure from Chen et al. ([2023b](#)) dramatically illustrates extrapolation failing:

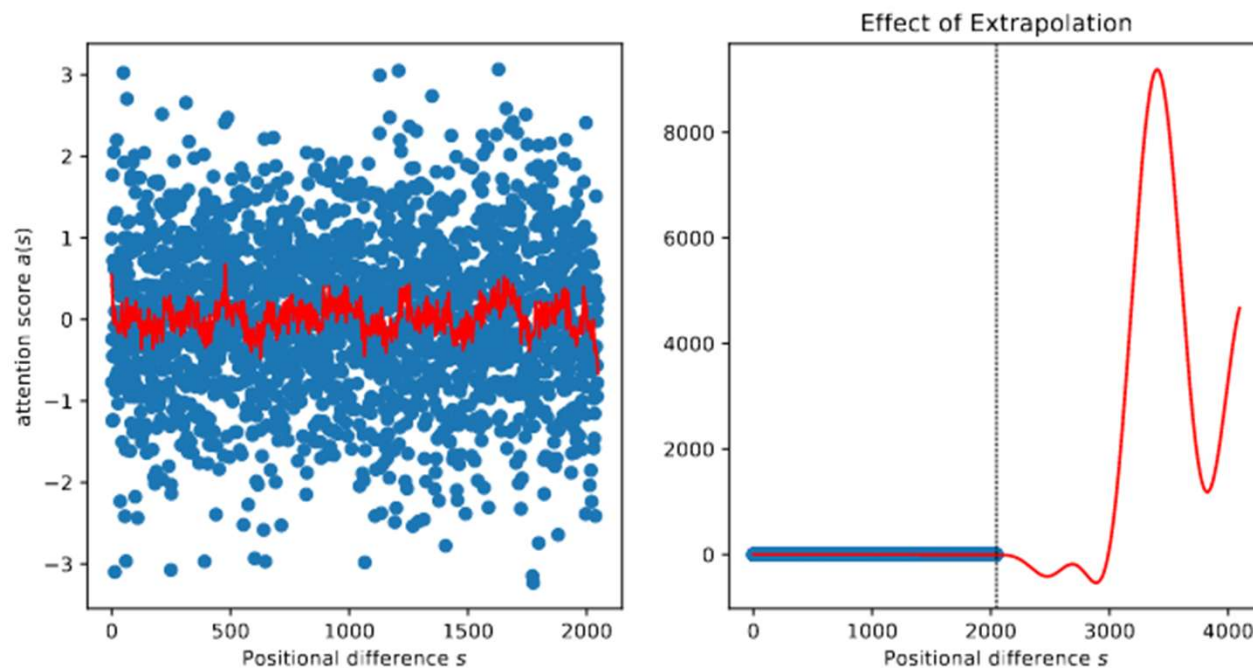


Figure 3: Left: A fitted attention score function (red curve) trained on positions $[0, 2048]$. Right: Extrapolation beyond the training range causes values to explode beyond 8000, breaking attention computation. (Figure 2 from Chen et al., 2023)

Evidence of Extrapolation Failure

The inability of Transformers to extrapolate beyond their training context is well-documented, as comprehensively summarized in [kaiokendev's analysis](#):

- Anil et al. ([2022](#)) demonstrated that several fine-tuning approaches fail to resolve length generalization pathologies, performing a comprehensive study showing multiple ways this problem manifests.
- Press, Smith, and Lewis ([2022](#)) found that Transformer models overfit to specific position embeddings seen during training, even with RoPE. They proposed ALiBi (Attention with Linear Biases) as a solution, observing that models essentially memorize position-token pairs rather than learning generalizable positional patterns.
- Liu et al. ([2023](#)) observed catastrophic glitches in long-range language modeling, with minor fluctuations in attention head logits causing complete failure beyond training lengths.
- Chi et al. ([2022](#)) analyzed position embeddings through receptive field analysis, finding that constraining the receptive field can actually improve extrapolation.
- Tao, Feng, and Zhao ([2023](#)) discovered that rear position embeddings are updated less frequently than front positions during training, leading to poor generalization at longer contexts.

Main Idea of the next step

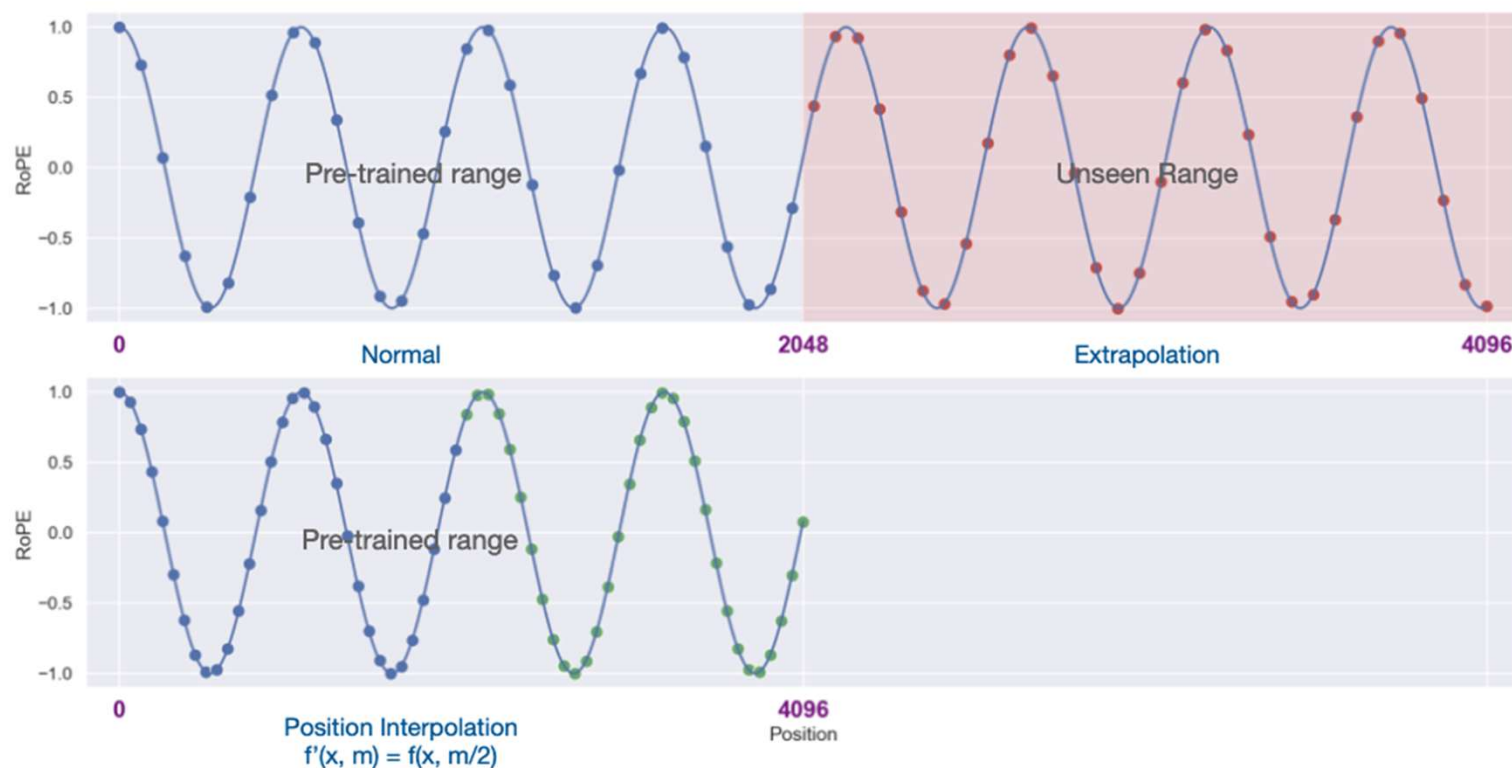
Position interpolation

Extends context by dividing every position (or frequency) by the extension factor s , which keeps all angles inside their trained range.

Downside: it also compresses the fast pairs, blurring the model's ability to tell neighboring tokens apart, so it needs substantial fine-tuning and degrades beyond certain extension ranges (roughly $8\times$).

Interpolation instead of Extrapolation

The fundamental realization is beautifully simple: instead of asking the model to handle positions it's never seen (extrapolation), we compress longer sequences to fit within the position range it knows (interpolation).



Interpolation instead of Extrapolation

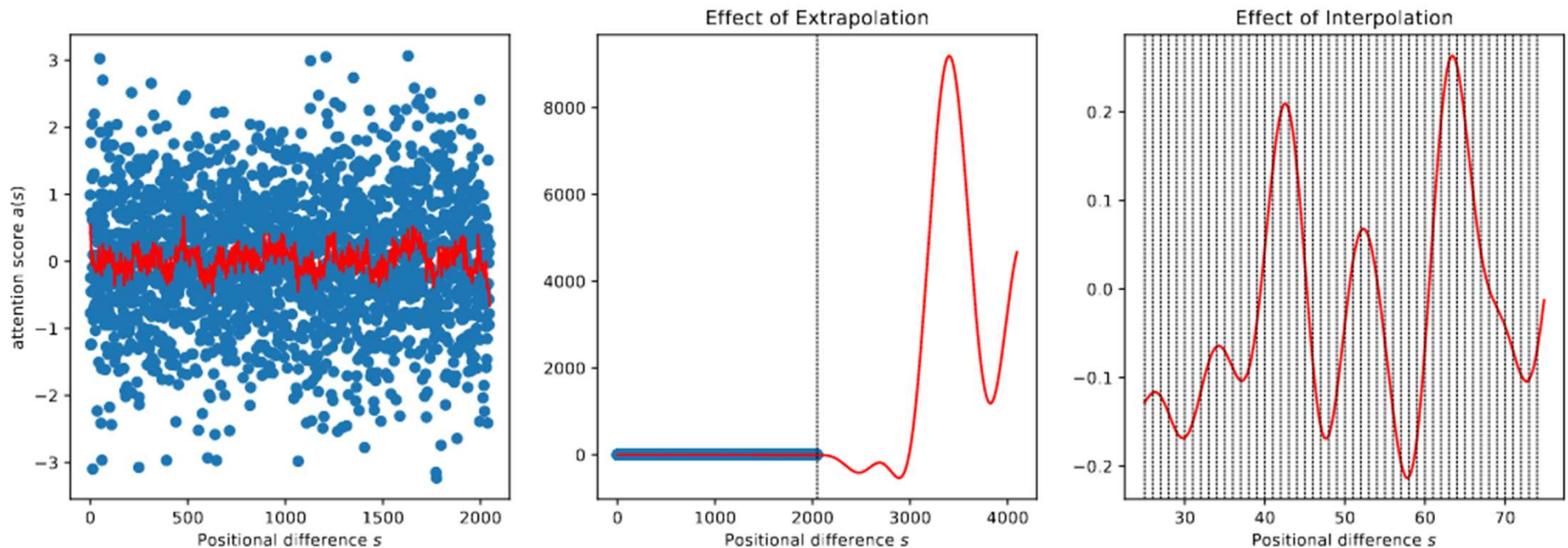
- We reduce position indices from $[0, L')$ to $[0, L)$ to match the original range of indices before computing RoPE.
- This transformation on the position encoding is called **Position Interpolation**.
- In simpler terms, during inference with context length $L_{context} > L_{train}$, we scale position indices:

$$\text{position}_{interpolated} = \frac{\text{position} \cdot L_{train}}{L_{context}}$$

- This ensures all positions map to the range $[0, L_{train}]$ that the model saw during training.

Evidence of Extrapolation Failure

The following figure from Chen et al. ([2023b](#)) dramatically illustrates extrapolation failing:



Key observations:

- Left panel:** The attention score function learned during training appears well-behaved within $[0, 2048]$
- Middle panel:** Beyond the training range, the function explodes to values over 8000 - causing catastrophic failure in attention computation
- Right panel:** With interpolation, positions are compressed to stay within the training range, keeping the function stable and well-behaved

Position interpolation: Squeeze

$$s = \frac{L_{new}}{L} \quad \text{e.g.} \quad s = \frac{8192}{4096} = 2$$

- Replace $f(x, m)$ with $f\left(x, \frac{m}{s}\right)$.
 - m is the token's position index in the sequence (0, 1, 2, ...), the same m as in the rotation angle $m\theta_i$.
 - **$f(x, m)$ is the RoPE function:** take a query or key vector x and rotate it by the angles $m\theta_i$ for its position m .
 - **Position interpolation replaces this with $f(x, m/s)$:** the vector for the token at position m is rotated as if it sat at position m/s instead.
 - So a token at position 8192 in a model trained to length 4096, with $s = 2$, gets the rotation of position 4096, the largest angle the model has seen.
 - Since the angle is $m\theta_i$, dividing m by s is identical to dividing every θ_i by s , which is why we now write the same operation as $\theta_i \rightarrow \frac{\theta_i}{s}$.
- Since the angle is $m \cdot \theta_i$, this is the same as $\theta_i \rightarrow \frac{\theta_i}{s}$ for every pair.

Position interpolation: Limitation

The price is resolution:

Adjacent tokens used to differ by θ_i ; now they differ by $\frac{\theta_i}{s}$.

- Take pair 0 with $s = 8$: gap 1 and gap 2 used to be 1 radian apart, a big, easily read difference;
- Now they are 0.125 radians apart, which is what a gap of one eighth of a token used to look like.
- The fast pairs were the model's ruler for local order (subject before verb, token inside a word), and interpolation shrinks the tick marks by s .
- The PI paper proved that the interpolated scores stay within a bound roughly 600 times tighter than extrapolated ones, which is why it works at all, but it needs fine-tuning to relearn the shrunken ruler, and beyond about 8x, the local damage shows.

Main Idea of the next step

NTK-aware scaling

Extends context by increasing the magnitude of RoPE's base so that the fast pairs are left almost untouched, and only the slow pairs are compressed, preserving fine positional detail while still keeping the slow angles in range.

Downside: the compression schedule follows pair index rather than actual training coverage, so some slow-but-not-slowest pairs still extrapolate slightly out of range, and some mid pairs get compressed for no benefit; it works without fine-tuning only up to modest s .

The Problem with Linear Interpolation

- Position Interpolation has a fundamental issue: when you compress positions linearly, adjacent tokens become harder to distinguish.
 - For example, with a 4x compression:
 - Original positions: 100, 101, 102, 103
 - After interpolation: 25.0, 25.25, 25.5, 25.75
- The compressed positions are so close that the model struggles to maintain the fine-grained distinctions it learned during training. This becomes catastrophic at higher compression ratios.

Trick

- Achieve the differential slow-down by making the base b bigger.
 - How much bigger?
 - A function of s .

The NTK Insight

Instead of scaling positions, NTK-aware scaling modifies the RoPE base frequency:

$$\text{base}_{\text{new}} = \text{base} \times s^{\frac{d}{d-2}}$$

where:

- s is the context extension factor (e.g., 8 for 2K→16K extension),
- d is the hidden dimension
- base is the original base (typically 10000)

But why change the base?

- Recall that RoPE frequencies are computed as:

$$\theta_i = \text{base}^{-\frac{2i}{d}}$$

- By increasing the base, we **slow down all rotation frequencies proportionally**. This is fundamentally different from position interpolation:
- **Position Interpolation**: Compress all frequencies, by a fixed factor
- **NTK Scaling**: Keep position indices, adjust rotation frequencies

NTK-aware: one base change that scales each pair differently

- **The insight:** pairs 0 to 45 needed no help; only the slow ones did, and PI squeezed all 64.
 - Uniform squeezing throws away high-frequency information, which neural networks are already bad at learning from compressed inputs
 - that is the **NTK theory** result the name refers to.
- **The fix:** If we increase the magnitude of the base b by multiplying it with a number β , we effectively slow down the frequencies.
 - Let β be a function of s so that we slow down proportionally based on the new context.

- Change the base 10000 to $b' = 10000 \cdot \beta = 10000 \cdot s^{\frac{d}{d-2}}$. Then

$$\theta'_i = b'^{-\frac{2i}{d}} = \theta_i \cdot s^{-\frac{2i}{d-2}} = 10000^{-\frac{2i}{d}} \cdot s^{-\frac{2i}{d-2}}$$

NTK-aware: one base change that scales each pair differently

$$\theta'_i = b'^{-\frac{2i}{d}} = \theta_i \cdot s^{-\frac{2i}{d-2}}$$

Read the exponent:

- At $i = 0$ it is 0, so the fastest pair is untouched.
- At $i = \frac{d}{2} - 1$ it is $-\frac{d-2}{d-2} = -1$, so the slowest pair gets exactly $\frac{\theta}{s}$, full interpolation.
- Every pair in between is interpolated by a geometric fraction of s .
- The odd-looking exponent $\frac{d}{d-2}$ is there purely so the last pair lands on exactly $\frac{1}{s}$.
- Its flaw: the schedule is set by the index i , not by r_i .
 - Pairs near the slow end, other than the last one, get squeezed by less than s while their wavelength is still longer than L , so they still poke slightly off-arc.
 - And mid pairs that completed dozens of turns still get squeezed a little for no reason.

The NTK Insight

The following graph from the original Reddit post ([bloc97 2023b](#)) shows the dramatic improvement of NTK-aware scaling over linear interpolation:

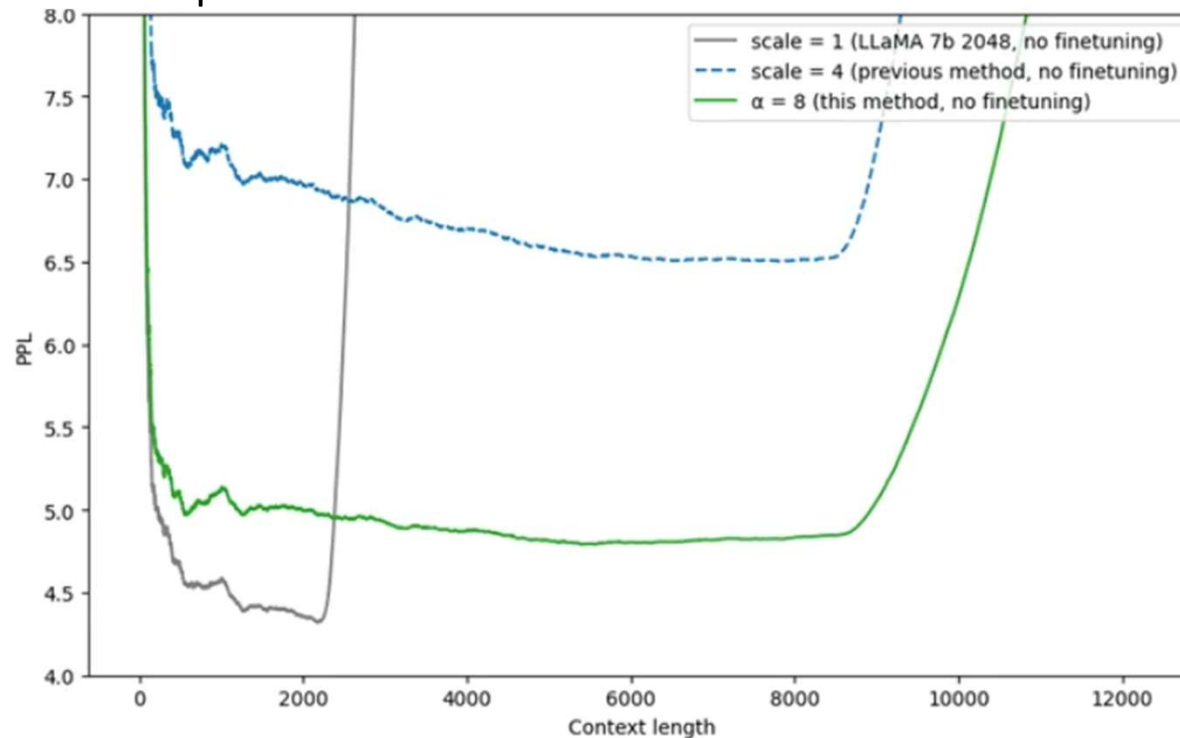


Figure 4: Perplexity comparison of different context extension methods on LLaMA 7B. Gray line: baseline (scale=1), Blue dashed: linear interpolation (scale=4), Green solid: NTK-aware scaling ($\alpha=8$). NTK-aware scaling maintains much lower perplexity across extended context lengths without any fine-tuning.

The Fixed Scaling Dilemma

Both Position Interpolation and NTK-aware scaling require choosing a fixed scaling factor upfront:

- **Choose a large factor:** Good for long sequences, but degrades short-sequence performance
- **Choose a small factor:** Preserves short sequence quality, but limits extension capability

This forces an unnecessary compromise before you even know what sequence lengths you'll process.

Dynamic Linear Scaling

The key insight: **Use exact positions for the first 2K (i.e. L) tokens, then scale only as needed:**

This means for example:

- **Positions 0-2048:** Use exact trained positions (scale=1.0)
- **Position 4096:** scale=2.0
- **Position 8192:** scale=4.0

The model uses its original training for short sequences and smoothly transitions to interpolation only when necessary.

Dynamic Linear Scaling

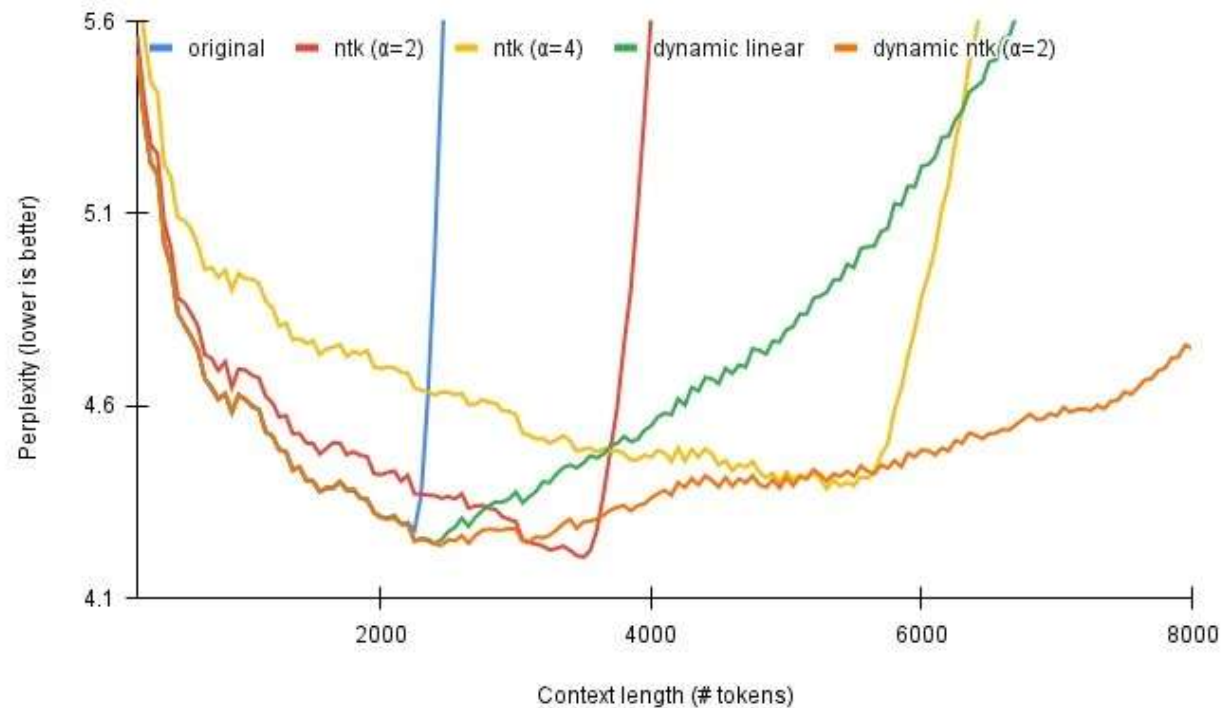


Figure 5: Perplexity comparison of dynamic scaling methods on LLaMA. Dynamic NTK (orange) achieves the best overall performance, maintaining low perplexity across all context lengths without any fine-tuning. Note how all dynamic methods avoid the catastrophic failure of static methods.

Main Idea of the next step

NTK-by-parts

Does the same thing as NTK-aware pair-by-pair by pair, fully compressing pairs that never completed a turn in training, leaving pairs that completed many turns alone, and blending in between.

Downside: the two thresholds are empirical, the blended middle pairs are still partly off-distribution, and on its own it does nothing about **attention** getting diluted over s times more keys.

Advanced: NTK “By Parts” - Frequency-Aware Interpolation

After the initial NTK-aware scaling, bloc97 discovered that different frequency components benefit from different scaling strategies. This led to the “by parts” correction: a more sophisticated approach that applies different interpolation methods to different parts of the frequency spectrum.

The Multi-Scale Problem

The key insight: RoPE’s frequency components encode information at different scales:

- **High frequencies** (low dimensions): Encode local, fine-grained relationships
- **Low frequencies** (high dimensions): Encode global, long-range dependencies

Applying the same scaling strategy to all frequencies is suboptimal.

- High frequencies should use linear interpolation to preserve local patterns, while
- Low frequencies benefit from NTK scaling for long-range coherence.

How it works

NTK-by-parts sorts RoPE pairs by how many rotations they completed at the training length.

- Pairs with many rotations (high frequency, local order) already know the whole circle and are left at their original frequencies.
- Pairs with fewer than one rotation (low frequency, long range) are the ones that would go out of range, so they are fully interpolated by s .
- Pairs in between get a linear blend.

This keeps local resolution intact while pulling the long-range hands back onto their trained arc, which is why it avoids both PI's blur and NTK-aware's late blowup.

- Setting the blend factors to zero recovers plain interpolation, and YaRN is this rule plus a temperature.

Remember r_i

$$r_i = \frac{\text{number of tokens in training}}{\text{number of tokens in one full turn}} = \frac{L}{\lambda_i} = L \cdot \frac{\theta_i}{2\pi}$$

*That is, how many turns pair i completed **during training**.*

NTK-by-parts: decide by turns, not by index

Use r_i directly. Pick two thresholds, $\alpha = 1$ turn and $\beta = 32$ turns, and a ramp:

$$\gamma_i = \begin{cases} 0 & r_i < \alpha \quad (\text{less than 1 turn, low frequency}) \\ \frac{r_i - \alpha}{\beta - \alpha} & \alpha \leq r_i \leq \beta \quad (\text{between 1 and 32 turns}) \\ 1 & r_i > \beta \quad (\text{more than 32 turns, high frequency}) \end{cases}$$

New frequency:

$$\theta'_i = (\mathbf{1} - \boldsymbol{\gamma}_i) \frac{\boldsymbol{\theta}_i}{s} + \boldsymbol{\gamma}_i \boldsymbol{\theta}_i$$

- $r_i < 1$ (never completed a turn): $\gamma = 0$, full interpolation, because any extrapolation is off-arc.
- $r_i > 32$ (many turns): $\gamma = 1$, leave it alone, because it has seen every angle many times and squeezing it is pure loss.
- **Between 1 and 32:** blend linearly ($0 \leq \gamma_i \leq 1$).

Same shape of idea as NTK-aware, but the boundary is now placed exactly where the "did it see the circle" question changes answer.

Main Idea of the next step

YaRN

Combines **NTK-by-parts** with a small **logit sharpening factor** that grows with $\ln s$, restoring the attention peakiness that gets diluted by having s times more keys, then fine-tunes briefly.

Downside: the temperature is a single average correction that helps focused heads and mildly hurts diffuse ones; its constants were fitted on one model family and may not transfer, it still needs some fine-tuning.

YaRN

YaRN combines:

1. **NTK-by-parts interpolation:** Frequency-aware scaling from the previous section
2. **Attention temperature scaling:** Preserves local token distinctions

From the paper, this dual approach allows YaRN to:

- Extend context to 128K+ tokens with minimal perplexity degradation
- Maintain fine-grained positional discrimination
- Require only lightweight fine-tuning (often <1% of pretraining compute)

YaRN

YaRN changes nothing in the model weights and nothing in the attention kernel.

It edits only the RoPE cos/sin table, in two ways:

1. A per-pair frequency multiplier and
2. One global scalar.

Then it fine-tunes briefly.

Example:

- $d = 128$,
- $L = 4096$,
- extending $16 \times$, so $s = 16$,
- $L_{new} = 65536$

Part 1: NTK-by-parts (which hands to slow, and by how much)

Step 1. Score each pair by how much of the circle it saw.

$$r_i = \frac{L}{\lambda_i} = \frac{L \theta_i}{2\pi} \text{ (turns completed in training)}$$

Step 2. Turn that score into a mixing weight $\gamma_i \in [0, 1]$, with thresholds $\alpha = 1$ and $\beta = 32$:

$$\gamma_i = \begin{cases} 0 & r_i < 1 \\ \frac{r_i - 1}{31} & 1 \leq r_i \leq 32 \\ 1 & r_i > 32 \end{cases}$$

- Read γ_i as "how much this pair is allowed to keep its original speed."

Step 3. Mix the two extremes. Full interpolation is θ_i/s ; no change is θ_i :

$$\theta'_i = (1 - \gamma_i) \frac{\theta_i}{s} + \gamma_i \theta_i = \theta_i \cdot \underbrace{\left[\frac{1 - \gamma_i}{s} + \gamma_i \right]}_{\text{multiplier between } 1/s \text{ and } 1}$$

- Since angle = $m\theta_i$, scaling θ_i scales every angle of that pair by the same factor. Nothing else changes.

NTK-by-parts

- **See it on the example.** The two boundaries fall at $r = 32 \Rightarrow i \approx 21$ and $r = 1 \Rightarrow i \approx 45$.

Pair	r_i	γ_i	Multiplier	Effect
0	652	1	1	untouched
20	37	1	1	untouched
30	8.7	0.25	0.30 ($\approx 1/3.4$)	partly slowed
40	2.1	0.03	0.094 ($\approx 1/10.6$)	mostly slowed
63	0.075	0	1/16	fully slowed

- Check the slow end:
 - At position 65536, pair 63 reaches angle $65536 \cdot \frac{\theta_{63}}{16} = 4096 \theta_{63}$, exactly its training maximum.
- Check the fast end:
 - Pair 0 keeps its 1 rad per token, so local order is still read at full resolution.
- **Why $\alpha = 1$ and $\beta = 32$.**
 - Below one turn the pair only ever saw an arc, so any speed above $\frac{\theta_i}{s}$ sends it off-arc: it must be fully interpolated.
 - Above 32 turns the pair has wrapped so often that its learned behavior is fully periodic; slowing it buys nothing and costs resolution.
 - The ramp between avoids a cliff.
 - The 32 was chosen empirically.

Attention Scores got flatter!

1. More competitors. Softmax weight on the most relevant key is:

$$w = \frac{e^{z^*}}{e^{z^*} + \sum_{\text{others}} e^{z_j}}$$

- Extend the context by s and the denominator has $s \times$ more terms. Even if the model produces exactly the same logits, the right key's share falls because it is diluted by more near-zero competitors.
- **Toy:** one right key with logit 9, everyone else at 0.
 - Training, 4096 keys: $w = \frac{8103}{(8103+4096)} \approx 66\%$
 - Extended, 65536 keys: $w = \frac{8103}{8103+65536} \approx 11\%$
 - Same head, same logits, six times less focus.
 - The value vector it passes forward is now a blur of 65k keys instead of mostly one.
 - [Every layer above was trained on the 66% version.](#)

2. Slowed pairs make keys look alike. In the interpolated pairs, keys at different positions now differ by smaller angles, so their positional signatures cluster and their logits cluster with them. Smaller logit spread means a flatter softmax.

Solution

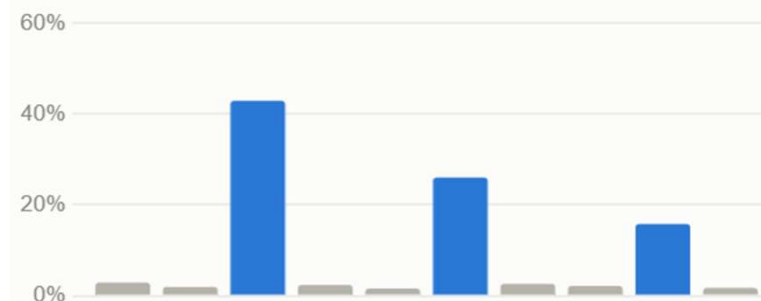
- The correction exists because the extension made the attention *flatter* than it was in training, and the later layers were trained on a certain peakiness.
- **The flattening was order-preserving, so the relevant scores are still numerically higher than the irrelevant ones.**
- **To make the distribution sharper just multiply every score with a number κ .**
- The magnitude of κ would be dictated by how aggressively we would like to regain peakiness. And that depends on s , i.e., how much more tokens have we added to the context.
- With $\kappa = \frac{1}{t} > 1$, we multiply every logit, which stretches the gaps between them, so softmax puts more mass on the top keys.

$$\text{softmax} \left(\kappa \cdot \frac{q_m \cdot k_n}{\sqrt{d}} \right), \kappa > 1 = \text{softmax} \left(\frac{1}{t} \cdot \frac{q_m \cdot k_n}{\sqrt{d}} \right), \frac{1}{t} > 1$$

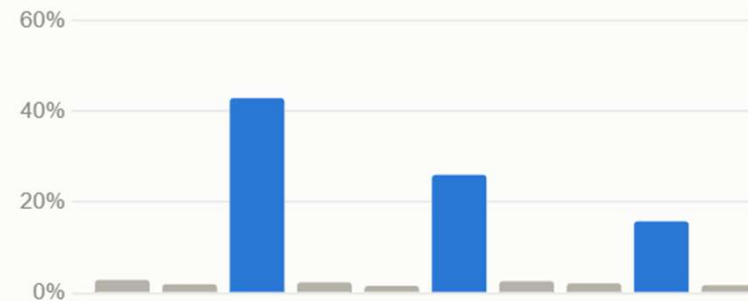
- Larger $1/t$ means sharper attention.

■ relevant token (logits 3.0, 2.5, 2.0) ■ distractor (logits near 0)

$N = 10, \kappa = 1$ (training) · relevant share 85%, top 43%



$N = 10$, no scaling needed · relevant share 85%, top 43%



$N = 30$ ($s = 3$), $\kappa = 1$ · relevant share 59%, top 30%



$N = 30$, $\kappa = 1.51$ · relevant share 85%, top 50%



$N = 60$ ($s = 6$), $\kappa = 1$ · relevant share 40%, top 20%



$N = 60$, $\kappa = 1.79$ · relevant share 85%, top 54%



Order is preserved throughout.

In every panel the blue bars keep their $3.0 > 2.5 > 2.0$ ranking and every distractor stays below them. Dilution changes heights, never ordering, because the logits themselves never changed; only the denominator grew.

Flattening is pure dilution.

The three relevant tokens hold 85% of the mass at 10 tokens, 59% at 30, and 40% at 60. The 20 extra distractors at $N = 30$ each take about 1.5%, but there are 20 of them, and that mass has to come from somewhere.

The restoring κ grows with s , slowly.

$\kappa = 1.51$ brings $N = 30$ back to 85%; $N = 60$ needs $\kappa = 1.79$. Doubling s from 3 to 6 added only 0.28 to κ , which is the logarithmic behavior behind YaRN's $0.1 \ln s + 1$ fit

Note the top token comes back at 50% and 54%, not 43%:

uniform sharpening cannot restore the exact original shape, only the share of the winners versus the crowd, which is why it is an average-case fix that fine-tuning then polishes.

Part 2: Attention temperature (restore the trained sharpness)

The problem. After extension, each query competes against $s \times$ more keys, and in the slowed pairs, nearby keys now look more alike than they did in training. Both flatten the pre-softmax distribution, so attention entropy is higher than what the later layers were trained to receive.

Step 1. The fix is a scalar on the logits.

$$\text{softmax} \left(\frac{1}{t} \cdot \frac{q_m \cdot k_n}{\sqrt{d}} \right), \frac{1}{t} > 1$$

- Larger $1/t$ means sharper attention.

Why does it make sense?

- The scores got bunched together, but the flattening was order-preserving
- Multiplying the scores with a number greater than 1, increases the mutual gaps between scores.

Part 2: Attention temperature (restore the trained sharpness)

The problem. After extension, each query competes against $s \times$ more keys, and in the slowed pairs, nearby keys now look more alike than they did in training. Both flatten the pre-softmax distribution, so attention entropy is higher than what the later layers were trained to receive.

Step 1. The fix is a scalar on the logits.

$$\text{softmax} \left(\frac{1}{t} \cdot \frac{q_m \cdot k_n}{\sqrt{d}} \right), \frac{1}{t} > 1$$

- Larger $1/t$ means sharper attention.

Step 2. Bake it into the RoPE cache so it is free. RoPE gives $q_m = R_m q$ and $k_n = R_n k$. Multiply the cos/sin table by $c = \sqrt{1/t}$, so every rotation matrix becomes cR :

$$(cR_m q) \cdot (cR_n k) = c^2 (R_m q) \cdot (R_n k) = \frac{1}{t} q_m \cdot k_n$$

- One constant in the table, no change to the attention code, works with fused kernels.

Step 3. Choose $1/t$. YaRN swept t across many s values and fit the best ones with

$$\sqrt{1/t} = 0.1 \ln(s) + 1$$

Softmax

- YaRN introduces a temperature parameter t on the logits before the attention softmax. Instead of the standard attention computation:

$$\text{softmax}\left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{\sqrt{D}}\right)$$

YaRN modifies it to:

$$\text{softmax}\left(\frac{\mathbf{q}_m^T \mathbf{k}_n}{t\sqrt{D}}\right)$$

where $t = \sqrt{1/s} = 0.1 \ln(s) + 1$ for scale factor s .

- The reparametrization of RoPE as 2D rotation matrices provides an elegant implementation.
- By scaling the complex RoPE embeddings by $\frac{1}{t}$, YaRN effectively alters the attention mechanism without modifying its code.
- This “length scaling” trick scales both q_m and k_n by a constant factor, achieving the temperature effect with zero overhead during inference.

Part 2: Attention temperature (restore the trained sharpness)

$$\sqrt{\frac{1}{t}} = 0.1 \ln(s) + 1$$

The log form means each doubling of s adds a fixed $0.1 \ln 2 \approx 0.07$ to $\sqrt{1/t}$:

- Sharpen a little, and sharpen more slowly as the context grows.
- This is an average-case correction; it does not recover resolution, it only returns the distribution to the peakiness the model expects.

s	$\sqrt{1/t}$	$1/t$
8	1.21	1.46
16	1.28	1.63
32	1.35	1.81

Question

Are these techniques applied after the model has already been trained using RoPE?

Yes. The original method starts from a pretrained RoPE model, changes the positional mapping, and then may continue training with that mapping. The modified mapping remains in use during inference.

Fine-tuning on what exactly?

On text sequences using ordinary next-token prediction, with the modified positional encoding active.

- The original methods work used a pretraining corpus such and extended context windows.

A concrete training procedure is:

1. Load the pretrained model.
2. Change RoPE to use $p/4$.
3. Feed long text sequences, for example up to 16,000 tokens.
4. Predict each next token from the preceding tokens.
5. Update the model's learned weights through backpropagation.

The training signal is the text itself. Human-written question–answer labels are unnecessary.

Main Idea of ALiBi

Skips rotation entirely and subtracts a per-head linear penalty proportional to distance from every attention logit, which extrapolates to any length for free but imposes a fixed recency bias that makes long-range retrieval hard.

Downside: the fixed recency bias caps long-range retrieval, offers only one monotone distance signal instead of a multi-frequency fingerprint, and gives up the relative-position richness that made RoPE the default.

Neural Tangent Kernel (NTK)

What is NTK?

- **Neural Tangent Kernel (Jacot et al., 2018).** For a very wide neural network trained by gradient descent, learning can be approximated by a fixed kernel method. Some patterns are naturally easy for the network to learn, so it learns them quickly. Other patterns are harder, so it learns them much more slowly.
- **Spectral bias.** For a plain MLP fed raw coordinates, such as a position number, the NTK tends to favor low-frequency functions. Smooth, slowly varying structure is learned easily, while fine, rapidly varying structure is much harder to learn.
- **The fix: Fourier features (Tancik et al., 2020).** Instead of feeding the coordinate directly, first encode it using sines and cosines at multiple frequencies. This changes the effective NTK bandwidth and makes higher-frequency structure much easier for the network to represent and learn.

NTK-Aware Methods

Why “NTK-aware”?

Tancik et al. used NTK theory to show that low-dimensional networks struggle with high-frequency structure unless sufficiently high-frequency Fourier features are provided. RoPE behaves like a Fourier feature encoding of position.

- **PI** scales *all* RoPE frequencies down, reducing the high-frequency positional detail useful for distinguishing nearby tokens.
- **NTK-aware scaling** therefore scales frequencies **unequally**: high frequencies are changed less, while low frequencies are slowed more to accommodate longer contexts.
- **NTK-by-parts** makes this explicit: preserve fast frequencies, interpolate slow frequencies, and smoothly blend those in between.

They are called “**NTK-aware**” **not because they compute an NTK**, but because their design was motivated by the NTK/Fourier-feature insight that **preserving high-frequency input features helps preserve fine positional resolution**.

NTK theory, in plain terms

- Think of a neural network as a painter, and the input features as the brushes it is handed. NTK theory studies what a very wide network can learn quickly under gradient descent, and its answer is: a network learns broad strokes fast and fine detail almost never, unless the fine detail is already present in its inputs. It cannot invent a fine brush. It can only combine the brushes it was given.
- Sines and cosines at many frequencies are exactly a set of brushes: slow waves are broad brushes, fast waves are fine brushes. Hand the network a fast wave and suddenly it can paint sharp detail, because the sharpness is now in the input, not something it has to discover.

Why this matters for RoPE

- RoPE's pairs are that brush set for position. The fast pairs are the fine brushes: they are what lets attention tell "the token right before me" from "the one two back."
- Position interpolation shrinks every brush by ss . The fine brushes become medium brushes, and the network is left with no way to paint the fine detail of local word order. Fine-tuning cannot fix this, because you cannot train a painter to draw hairlines with a roller.

Why the name

- "NTK-aware" means: aware that the network's resolution is capped by the finest brush you give it. So keep the fine brushes as they are, and only enlarge the broad brushes, which are the ones that actually run out of canvas when the context grows. NTK-aware does this with one base change; NTK-by-parts does it brush by brush, judged by how many turns each one completed in training.

What About the Eigen Directions?

Neural Tangent Kernel (Jacot et al., 2018). Take a very wide network and train it by gradient descent. Its behavior becomes equivalent to a fixed kernel method: the network learns the target function one eigen-direction of this kernel at a time, fast for large eigenvalues, slow for small ones.

- Think of the NTK as describing **which patterns the network is naturally good at changing/learning**.
- When we decompose this kernel mathematically, we get:
- **Eigen-directions** = basic patterns the network could learn.
- **Eigenvalue for a direction** = how strongly/easily the network can learn that pattern.

So:

- large eigenvalue $\Rightarrow$ learned quickly
- small eigenvalue $\Rightarrow$ learned slowly
- For position as a 1-D coordinate, these eigen-directions can roughly correspond to patterns like:
- slowly varying $\rightarrow$ low frequency $\rightarrow$ often **large eigenvalue**
- rapidly varying $\rightarrow$ high frequency $\rightarrow$ often **small eigenvalue**

So when we say:

- “the network learns the target function one eigen-direction at a time”
- it means that the desired function can be thought of as a **mixture of simple patterns**, and gradient descent learns each component at a rate determined by its eigenvalue.

The important connection is:

- **Eigenvalue = learning speed for that kind of pattern.**

That is why Fourier features matter: they reshape the kernel so that some high-frequency patterns that previously had tiny eigenvalues become much easier to learn.

We reshape it **at the input representation**, before the network.

Instead of giving the network the raw coordinate

$$x$$

we give it Fourier features:

$$\gamma(x) = [\sin(\omega_1 x), \cos(\omega_1 x) \dots \sin(\omega_k x), \cos(\omega_k x)]$$

That changes what two inputs “look like” to the network.

Originally, the kernel is effectively:

$$K(x, x')$$

After Fourier features, it becomes:

$$K(\gamma(x), \gamma(x'))$$

So the kernel itself changes because its **inputs have changed**.

Intuition

Suppose we want the network to learn a rapidly oscillating function.

With raw x :

$$x = 100, x = 101$$

are just two nearby scalar numbers. The network has to manufacture the rapid oscillation internally, which is difficult.

With a high-frequency feature:

$$\sin(\omega x), \cos(\omega x)$$

those two nearby positions may already map to **very different feature vectors**.

So the high-frequency variation is now **explicitly present in the input**. Consequently, the NTK assigns more learning capacity, i.e. larger effective eigenvalues, to those high-frequency patterns.

So:

We do not directly modify the kernel's eigenvalues. We modify the input using Fourier features, which indirectly changes the NTK and its eigenvalue spectrum.

And this is exactly the conceptual bridge to RoPE: its different-frequency sine/cosine rotations already provide such Fourier-like positional features.

Intuitively: If someone struggles to distinguish tiny differences, we **re-encode those differences into a form that is easier to notice**.

- For example, two nearly identical tiny objects may be hard to tell apart by size alone, but if one is red and one is blue, the distinction becomes obvious.
- Likewise, with raw position x , nearby positions can look very similar to the network. Fourier features turn those subtle positional differences into **larger, more distinctive patterns across sine/cosine features**, making them easier for the network to learn.

So the key idea is not “make the object bigger,” but:

- **make the important difference more visible in the representation.**